CMPE3209 Computer Organization — Proje Teslim Tutanağı

Kullanım talimatı

Bu tutanak projenin tek resmi teslim belgesidir. Her kilometre taşında (M1, M2 ve final; tümü pazartesi) ilgili adım doldurulur ve aynı dosya güncellenerek yeniden teslim edilir. Ayrıntılı görev tanımları proje yönergesindedir; bu tutanak yalnızca ne istendiğini ve nereye yazılacağını gösterir.

- Tüm sayılar birimiyle yazılır (byte, cycle, ns, GHz). Hex değerler `0x` önekiyle yazılır.
- **Varsayım** olan her değer "(varsayım)" etiketiyle işaretlenir.
- Tablolara sığmayan çıktılar (assembly listesi, objdump, CSV izi, ekran görüntüleri) Git deposuna konur. Tutanakta dosya adıyla atıf yapılır.
- Boş bırakılan zorunlu alan, o adımın puanını sıfırlar.

Öğrenci bilgileri

Alan	Değer
Ad Soyad	Yusuf Mert Turan
Öğrenci numarası (SID)	5001240003 (programda son 9 hane: 1240003)
E-posta	yusufmertturan@gmail.com
Seçilen görev (G1–G5)	G4 Sıralama ve medyan
Seçilen ISA ve sürümü (ör. RV32IM)	RISC-V RV32IM (-march=rv32im -mabi=ilp32). Gerekçe: yönergenin önerdiği ISA; sabit 32 bit instruction uzunluğu ve düzenli formatlar (R, I, S, B, U, J) elle kodlamayı kolaylaştırır; M uzantısı çarpmayı (Y4) tek instruction olarak sağlar; RARS ve riscv64-elf-gcc ile araç zinciri hazırdır.
Derleyici ve sürümü	riscv64-elf-gcc 15.2.0 (GCC), -O0 ve -O1
Simülatör / araçlar	RARS 1.6, riscv64-elf-objdump, tools/gcc2rars.py (GCC çıktısını RARS'a çeviren betik)
Git deposu adresi	https://github.com/Jopseps/computer-organisation-project

Adım 0 — Kişiye özel parametreler

Bu adım M1 ile birlikte teslim edilir. Hatalı bir parametre, ona dayanan tüm adımları geçersiz kılar; öğretim üyesi bu tabloyu M1'de kontrol eder.

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
d	SID'in son hanesi	5001240003 → son hane 3	3
D2	SID'in son iki hanesi	son iki hane 03	03 (= 3)
S	SID'in hanelerinin toplamı	5+0+0+1+2+4+0+0+0+3 (tam	15

Sembol	Tanım	Hesap (adımlarıyla)	Sonuç
		numaranın 10 hanesi)	
N	24 + (D2 mod 25)	24 + (3 mod 25) = 24 + 3	27
Cache	d'ye göre: boyut / line / mapping / line sayısı / set sayısı	d = 3 satırı	256 B / 32 B line / 2-way set-assoc. / 8 line / 4 set
Write policy	d çift → write-back, d tek → write-through	d = 3 tek	write-through
k	(S mod 7) + 3	(15 mod 7) + 3 = 1 + 3	4
Interrupt türü	S çift → Timer, S tek → I/O	S = 15 tek	I/O
f	1.0 + 0.1 × d GHz (varsayım)	1.0 + 0.1 × 3	1.3 GHz (varsayım)
T1 / T2	T1 = 1 cycle; T2 = 20 + 5 × (d mod 4) cycle (varsayım)	T1 = 1; T2 = 20 + 5 × (3 mod 4) = 20 + 15	T1 = 1 cycle; T2 = 35 cycle (varsayım)
Nc	2 + (S mod 7) (varsayım)	2 + (15 mod 7) = 2 + 1	3 (varsayım)
Görev parametresi	Seçilen göreve göre (T, P, K, yön veya R)	G4: SID (1240003) tek → azalan	Sıralama yönü: azalan

Adres alanları (ISA'nın adres genişliği s + w bit; 64-bit ISA'da düşük 32 bit kullanılır):

Alan	Bit sayısı	Hesap
Word (w)	5	$\log_2$(32 B line) = 5
Line (r) veya set (d)	2	set (d): $\log_2$(4 set) = 2
Tag (s − r veya s − d)	25	tag = 32 − 2 − 5
Toplam	32	25 + 2 + 5 = 32 (RV32, 32-bit adres)

Adım 1 — Görev seçimi ve C programı (M1: 2 Kasım, Pazartesi)

1.1 Görev seçimi (yalnızca birini işaretleyin):

- ☐ G1 Biyomedikal sinyal alarmı
- ☐ G2 Sınav notu dağılımı
- ☐ G3 Mesaj şifreleme ve bütünlük
- ☒ G4 Sıralama ve medyan
- ☐ G5 Run-length sıkıştırma

1.2 Çözüm tasarımı (3–5 cümle: hangi algoritma, hangi fonksiyonlar, hangi veri yerleşimi ve neden):

> Program, N = 27 elemanlı global arr[] dizisini SID'den başlayan LCG ile doldurur (R = 1000, B = 0). Dizi, iç içe iki döngüyle yerinde bubble sort ile sıralanır; SID tek olduğu için yön azalandır. Karşılaştırma should_swap(a, b) fonksiyonunda yapılır; böylece kernel'de fonksiyon çağrısı ve dönüşü görünür, yer değiştirme sayısı global swaps değişkeninde sayılır. Bubble sort seçildi, çünkü basittir, yer değiştirme sayısı kolay izlenir ve arr[] üzerinde çok sayıda load/store üretir. Sıralamadan sonra medyan arr[(N−1)/2] elemanından en büyük ve en küçük değer uçlardaki elemanlardan okunur ve global değişkenlerde tutulur. Yer değiştirme, geçici değişken olmadan XOR ile yapılır (Y4 bit işlemi).

1.3 C kaynak kodu. Satır numaralı kod aşağıdaki bloka yapıştırılır (25–60 satır):

```
 1  /* Yusuf Mert Turan, 5001240003, G4 */
 2  #define N 27
 3
 4  const int SID = 1240003;
 5  int arr[N];
 6  int median, min_v, max_v, swaps;
 7
 8  int should_swap(int a, int b){
 9      if(a < b){
10          return 1;
11      }
12      return 0;
13  }
14
15  int main(){
16      unsigned int x = SID;
17      for(int i = 0; i < N; i++){
18          x = 1664525 * x + 1013904223;
19          arr[i] = (x >> 16) % 1000;
20      }
21
22      for(int i = 0; i < N - 1; i++){
23          for(int j = 0; j < N - 1 - i; j++){
24              if(should_swap(arr[j], arr[j + 1])){
25                  arr[j] ^= arr[j + 1];
26                  arr[j + 1] ^= arr[j];
27                  arr[j] ^= arr[j + 1];
28                  swaps++;
29              }
30          }
31      }
32
33      median = arr[(N - 1) / 2];
34      max_v = arr[0];
35      min_v = arr[N - 1];
36
37      return 0;
38  }
```

1.4 Zorunlu yapı eşlemesi. Her madde için satır numarası ve tek cümlelik açıklama:

#	Zorunlu yapı	Satır no	Açıklama
Y1	N elemanlı global int dizisi, en az iki geçiş	5, 17–20, 22–31	arr[N] global (N = 27); doldurma döngüsü (17–20) ve sıralama döngüleri (22–31) diziyi iki kez geçer.
Y2	Koşullu dallanma	9, 24	should_swap içindeki if

#	Zorunlu yapı	Satır no	Açıklama
			(9) ve sıralamadaki if (24) koşullu dallanmadır.
Y3	Parametreli, değer döndüren fonksiyon çağrısı	8–13, 24	should_swap(a, b) iki parametre alır, 0 veya 1 döndürür; satır 24'te çağrılır.
Y4	Toplama/çıkarma, çarpma veya shift, bit işlemi	18, 19, 25–27	Çarpma ve toplama (18), shift ve mod (19), bit işlemi XOR (25–27).
Y5	Dizi elemanına store	19, 25–27	Doldururken arr[i] (19); XOR ile yer değiştirirken arr[j] ve arr[j+1] yazılır (25–27).
Y6	const int SID ve hesapta kullanımı	4, 16, 18	const int SID = 1240003 (4); x = SID üretecin başlangıç değeri (16), dizinin tüm değerlerini etkiler (18).
Y7	Sonucun tutulduğu global değişken(ler)	6	median, min_v, max_v, swaps global değişkenlerdir (6); sonuçlar 28 ve 33–35. satırlarda yazılır.

1.5 Program çıktıları. Simülatörde çalıştırıldıktan sonra bellekten okunan değerler:

Global değişken	Adres (hex)	Değer	Ekran görüntüsü dosyası
median	0x10010004	494	build/rars_O0_result.txt (RARS komut satırı çıktısı)
min_v	0x10010008	4	build/rars_O0_result.txt (RARS komut satırı çıktısı)
max_v	0x1001000C	953	build/rars_O0_result.txt (RARS komut satırı çıktısı)
swaps	0x10010010	173	build/rars_O0_result.txt (RARS komut satırı çıktısı)

1.6 İzlenecek kernel (12–20 dinamik instruction; en az bir load, store, ALU, koşullu branch ve call/return içermeli):

Alan	Değer
C satır aralığı	24 (çağrı ve if), 8–13 (should_swap)
Başlangıç ve bitiş adresi (hex)	0x40012C – 0x400134 (arada should_swap: 0x400000 – 0x400040)
Dinamik instruction sayısı	18
Hangi döngü iterasyonu (ör. i = 3)	i = 0, j = 0 (arr[0] = 575, arr[1] = 383; yer değiştirme yok). Liste: kernel_M1.md

Öğretim üyesi (M1): ☐ Onaylandı ☐ Düzeltme istendi ☐ Reddedildi — Tarih: _____ Not: _____

Adım 2 — Assembly ve machine code (M2: 9 Kasım, Pazartesi)

2a. C → assembly eşlemesi (`-O0`; listenin tamamı depoda `asm_O0.s`):

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
8	int should_swap(int a, int b){	addi sp,sp,-32 … sw a1,-24(s0) (6 instr.)	0x400000–0x400014
9	if(a < b){	lw a4,-20(s0) … bge a4,a5,40002c <should_swap+0x2c> (3 instr.)	0x400018–0x400020
10	return 1;	li a5,1 … j 400030 <should_swap+0x30> (2 instr.)	0x400024–0x400028
12	return 0;	li a5,0	0x40002C–0x40002C
13	}	mv a0,a5 … ret (5 instr.)	0x400030–0x400040
15	int main(){	addi sp,sp,-32 … addi s0,sp,32 (4 instr.)	0x400044–0x400050
16	unsigned int x = SID;	lui a5,0x12f … sw a5,-20(s0) (3 instr.)	0x400054–0x40005C
17	for(int i = 0; i < N; i++){	sw zero,-24(s0) … j 4000d4 <main+0x90> (2 instr.)	0x400060–0x400064
18	x = 1664525 * x + 1013904223;	lw a4,-20(s0) … sw a5,-20(s0) (8 instr.)	0x400068–0x400084
19	arr[i] = (x >> 16) % 1000;	lw a5,-20(s0) … sw a3,0(a5) (16 instr.)	0x400088–0x4000C4
17	for(int i = 0; i < N; i++) { (artırma ve koşul)	lw a5,-24(s0) … bge a5,a4,400068 <main+0x24> (6 instr.)	0x4000C8–0x4000DC
22	for(int i = 0; i < N - 1; i++)	sw zero,-28(s0) … j 400264	0x4000E0–0x4000E4

C satır no	C ifadesi	Assembly satırları	Adres aralığı (hex)
	{	<main+0x220> (2 instr.)	
23	for(int j = 0; j < N - 1 - i; j++){	sw zero,-32(s0) … j 400244 <main+0x200> (2 instr.)	0x4000E8–0x4000EC
24	if(should_swap(arr[j], arr[j + 1])){	lui a5,0x10010 … beqz a5,400238 <main+0x1f4> (18 instr.)	0x4000F0–0x400134
25	arr[j] ^= arr[j + 1];	lui a5,0x10010 … sw a4,0(a5) (20 instr.)	0x400138–0x400184
26	arr[j + 1] ^= arr[j];	lw a5,-32(s0) … sw a4,0(a5) (21 instr.)	0x400188–0x4001D8
27	arr[j] ^= arr[j + 1];	lui a5,0x10010 … sw a4,0(a5) (20 instr.)	0x4001DC–0x400228
28	swaps++;	lw a5,-2032(gp) … sw a4,-2032(gp) (3 instr.)	0x40022C–0x400234
23	for(int j = 0; j < N - 1 - i; j++){ (artırma ve koşul)	lw a5,-32(s0) … blt a4,a5,4000f0 <main+0xac> (8 instr.)	0x400238–0x400254
22	for(int i = 0; i < N - 1; i++) { (artırma ve koşul)	lw a5,-28(s0) … bge a5,a4,4000e8 <main+0xa4> (6 instr.)	0x400258–0x40026C
33	median = arr[(N - 1) / 2];	lui a5,0x10010 … sw a4,4(a5) (5 instr.)	0x400270–0x400280
34	max_v = arr[0];	lui a5,0x10010 … sw a4,-2036(gp) (4 instr.)	0x400284–0x400290
35	min_v = arr[N - 1];	lui a5,0x10010 … sw a4,-2040(gp) (4 instr.)	0x400294–0x4002A0
37	return 0;	li a5,0	0x4002A4–0x4002A4
38	}	mv a0,a5 … ret (5 instr.)	0x4002A8–0x4002B8

Bölüm	Başlangıç adresi (hex)	Boyut (byte)	İçerik
.text	0x00400000	700 (0x2BC)	should_swap (0x400000), main (0x400044)
.data / .bss	0x10010000	128 (0x80)	SID (0x10010000); median, min_v, max_v, swaps (0x10010004–0x10010013); arr (0x10010014–0x1001007F)

Von Neumann ilkesi bu adreslerde nasıl görünüyor? (1–2 cümle)

> Komutlar (.text, 0x00400000'dan) ve veri (.sdata/.bss, 0x10010000'dan) aynı bellekte ve aynı adres uzayında duruyor; işlemci ikisini de aynı bus üzerinden okuyor. Yani program da veri de ikilik sayı olarak saklanıyor (stored-program), komutu veriden ayıran şey sadece hangi adresten okunduğu ve nasıl kullanıldığı.

2b. `-O0` ve `-O1` karşılaştırması (kernel için):

Ölçü	-O0	-O1	Farkın nedeni
Kernel'deki instruction sayısı	33 (satır 24: 18 main'de, 15 should_swap içinde)	3 (lw, lw, bge)	-O0 should_swap'ı gerçekten çağırıyor: prologue/epilogue, parametrelerin stack'e yazılıp geri okunması ve dizi adresinin her seferinde yeniden hesaplanması. -O1 fonksiyonu satır içine alıp tek bge'ye indiriyor.
Load sayısı	8	2	-O0'da j ve parametreler stack'te yaşadığı için her kullanımda lw gerekiyor. -O1'de j ve dizi göstericisi register'da, sadece arr[j] ve arr[j+1] okunuyor.
Store sayısı	4	0	-O0 ra, s0, a0, a1'i stack'e yazıyor (prologue). -O1'de çağrı ve stack çerçevesi yok.
Kullanılan register sayısı	8 (a0, a1, a3, a4, a5, s0, sp, ra)	3 (a3, a4, a5)	-O1'de register allocation değişkenleri register'larda tutuyor, stack ve çağrı için ayrılan s0/sp/ra gerekmiyor.

2c. Elle machine code (en az 10 instruction; x86-64 için 8). Kullanılan tüm formatlar kapsanmalı.

#	Adres (hex)	Assembly	Format	Alanlar (ad = binary)	Machine code (hex)	Bellekteki byte sırası	objdump ile aynı mı?
1	0x400020	bge a4,a5,0x40002c	B	imm[12]=0, imm[10:5]=000000, rs2=01111 (a5), rs1=01110 (a4), funct3=101, imm[4:1]=0110, imm[11]=0, opcode=1100011 (offset = 0x40002C − 0x400020 = 12)	0x00F75663	63 56 F7 00	Evet
2	0x400040	ret (jalr x0,0(ra))	I	imm[11:0]=000000000000, rs1=00001 (ra), funct3=000, rd=00000 (zero), opcode=1100111	0x00008067	67 80 00 00	Evet
3	0x400054	lui a5,0x12f	U	imm[31:12]=00000000000100101111 (0x0012F), rd=01111 (a5), opcode=0110111	0x0012F7B7	B7 F7 12 00	Evet
4	0x400058	addi a5,a5,-1085	I	imm[11:0]=101111000011 (−1085), rs1=01111 (a5), funct3=000, rd=01111 (a5), opcode=0010011	0xBC378793	93 87 37 BC	Evet
5	0x40005C	sw a5,-20(s0)	S	imm[11:5]=1111111, rs2=01111 (a5), rs1=01000 (s0), funct3=010, imm[4:0]=01100, opcode=0100011 (imm = −20)	0xFEF42623	23 26 F4 FE	Evet
6	0x400068	lw a4,-20(s0)	I	imm[11:0]=111111101100 (−20), rs1=01000 (s0), funct3=010, rd=01110	0xFEC42703	03 27 C4 FE	Evet

#	Adres (hex)	Assembly	For mat	Alanlar (ad = binary)	Machine code (hex)	Bellekteki byte sırası	objdump ile aynı mı?
				(a4), opcode=0000011			
7	0x400074	mul a4,a4,a5	R	funct7=0000001, rs2=01111 (a5), rs1=01110 (a4), funct3=000, rd=01110 (a4), opcode=0110011	0x02F70733	33 07 F7 02	Evet
8	0x4000FC	slli a5,a5,2	I	imm[11:5]=0000000, shamt=00010 (2), rs1=01111 (a5), funct3=001, rd=01111 (a5), opcode=0010011	0x00279793	93 97 27 00	Evet
9	0x40012C	jal ra,0x400000	J	imm[20]=1, imm[10:1]=1101101010, imm[11]=1, imm[19:12]=11111111, rd=00001 (ra), opcode=1101111 (offset = 0x400000 − 0x40012C = −300)	0xED5FF0EF	EF F0 5F ED	Evet
10	0x40016C	xor a4,a4,a5	R	funct7=0000000, rs2=01111 (a5), rs1=01110 (a4), funct3=100, rd=01110 (a4), opcode=0110011	0x00F74733	33 47 F7 00	Evet

Zorunlu instruction'ların tablodaki sıra numarası:

- ☒ SID'i yükleyen instruction(lar): # 3 ve 4
- ☒ Koşullu branch: # 1
- ☒ Load: # 6
- ☒ Store: # 5
- ☒ Fonksiyon çağrısı: # 9

SID'in yüklenmesi. Sabitin neden tek instruction'a sığmadığı, nasıl bölündüğü ve gerekiyorsa +1 düzeltmenin sign extension ile açıklaması:

> *Sabit 1240003 = 0x0012EBC3, 32 bit. RISC-V'te tek bir instruction'ın içine en fazla 20 bit (lui) veya 12 bit (addi) sabit sığdığı için iki instruction'a bölünüyor: lui üst 20 biti, addi alt 12 biti yükler. Üst 20 bit 0x0012E, alt 12 bit 0xBC3 (= 3011). addi'nin 12 bitlik sabiti işaretli (sign extension) okunuyor: 0xBC3'ün en üst biti 1 olduğu için değer −1085 (3011 − 4096) oluyor. Bu negatif toplamı dengelemek için lui'ye 0x12E yerine 0x12F yüklenir: 0x12F000 + (−1085) = 1241088 − 1085 = 1240003 = 0x12EBC3. Yani +1 düzeltmesi, alt yarının negatif okunmasından kaynaklanıyor.*

Branch offset hesabı. Branch'in adresi, hedef adres ve offset'in adım adım hesabı:

> *bge a4,a5 instruction'ı 0x00400020 adresinde, atlama hedefi 0x0040002C. Offset = hedef − instruction adresi = 0x2C − 0x20 = 12. B formatı 13 bitlik offset taşır (bit 0 her zaman 0 olduğu için saklanmaz): 12 = 0000000001100. imm[12]=0, imm[11]=0, imm[10:5]=000000, imm[4:1]=0110. Bu bitler formatın istediği sırayla (imm[12], imm[10:5], rs2, rs1, funct3, imm[4:1], imm[11], opcode) yerleştirilince 0x00F75663 elde edilir.*

YZ kullanım beyanı

YZ kullanım beyanı. Kullanılmadıysa ilk satıra "Kullanılmadı" yazın. Beyan edilmemiş kullanım akademik dürüstlük ihlalidir.

Araç ve sürümü	Adım	Ne için kullanıldı	Örnek prompt	Çıktıyı nasıl doğruladım
Claude Code (Claude Sonnet 5.5)	Adım 0	Parametrelerin hesaplanması ve açıklanması; S'nin tam numaradan mı 9 haneden mi hesaplanacağı belirsizliğinin tespiti	So why do we only use 9 digit numbers? Is it because of the 10 bit thing?	Formüller yönergeyle karşılaştırıldı, Python ile yeniden hesaplandı
Claude Code; RARS 1.6; riscv64-elf-gcc 15.2.0	Adım 1 (1.5, 1.6)	Araç kurulumu, GCC çıktısını RARS'a çeviren betik (tools/gcc2rars.py), kernel seçimi	Aight explain the rars stuff and give me docs and stackoverflows about it.	Adresler objdump ile, bellek değerleri RARS ile karşılaştırıldı
Claude Code (Claude Sonnet 5.5)	Adım 1 (1.2, 1.3, 1.4)	C kodunun kontrolü ve tutanak tablolarının doldurulması	Aight check this code and complete if it has missing stuff. Tell me what you changed.	Kodu okudum ve anladım, değişkenleri ve yorum satırlarını kendim düzenledim; derleyip RARS'ta çalıştırdım, sonuçları (494/4/953/173) kontrol ettim
Claude Code (Claude Sonnet 5.5)	Adım 1 düzeltme, Adım 2 (2a, 2b)	M1 geri bildirimine göre düzeltmeler (SID, değişken adları, & 0x3FF yerine XOR ile takas); C satırı–assembly eşlemesi ve -O0/-O1 tablolarının hazırlanması; RISC-V instruction formatlarının anlatımı; 2c: instruction formatlarını ve adımları Claude anlattı, boşluk doldurma şablonlarıyla yönlendirdi ve sonuçları kontrol etti (10 instruction'ın hex değerlerini öğrenci	Şöyle bir mail geldi hocadan: ... Ayrıca M2 teslimi ne zaman?	Sonuçlar RARS'ta yeniden çalıştırıldı ve hocanın verdiği değerlerle (494/4/953/173) karşılaştırıldı; eşleme addr2line ve objdump ile kontrol edildi; 2c: 10 hex değeri objdump ve tools/decode_rv.py ile karşılaştırıldı, hepsi aynı

Araç ve sürümü	Adım	Ne için kullanıldı	Örnek prompt	Çıktıyı nasıl doğruladım
		kendisi hesapladı); 2c açıklama paragraflarının taslağı		